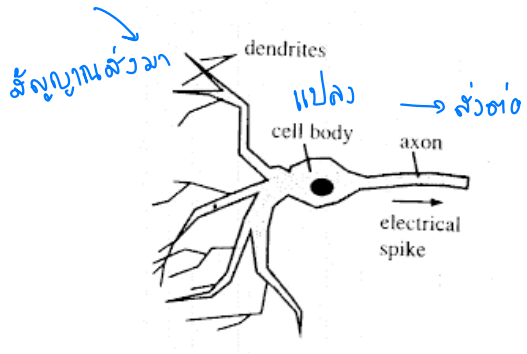
6.7 ข่ายงานประสาทเทียม

ข่ายงานประสาทเทียม (Artificial Neural Network) เป็นการจำลองการทำงานบางส่วนของสมองมนุษย์ เซลล์ประสาท (neuron) ในสมองของคนเราประกอบด้วยนิวเคลียส (nucleus) ตัวเซลล์ (cell body) ใยประสาทนำเข้า (dendrite) แกนประสาทนำออก (axon) แสดงใน รูปที่ 6–34



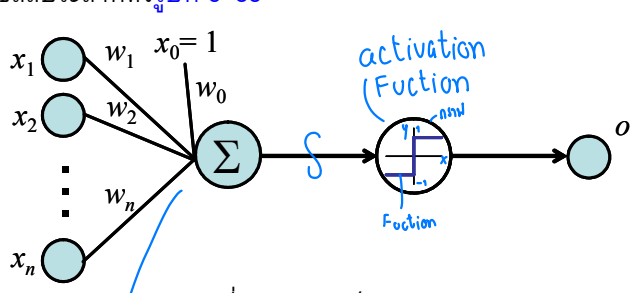
รูปที่ 6–34 เซลล์ประสาท

เดนไดรท์ทำหน้าที่รับสัญญาณไฟฟ้าเคมีซึ่งส่งมาจากเซลล์ประสาทใกล้เคียง เซลล์ประสาทตัวหนึ่งๆ จะเชื่อมต่อกับเซลล์ตัวอื่นๆ ประมาณ 10,000 ตัว เมื่อสัญญาณไฟฟ้าเคมีที่รับเข้ามาเกินค่าค่าหนึ่ง เซลล์จะถูกกระตุ้นและส่งสัญญาณไปทางแกนประสาทนำออกไปยังเซลล์อื่นๆ ต่อไป ประมาณกันว่าสมองของคนเรามีเซลล์ประสาทอยู่ทั้งสิ้นประมาณ 10^{11} ตัว

6.7.1 เพอร์เซปตรอน

เพอร์เซปตรอน (perceptron) เป็นข่ายงานประสาทเทียมแบบง่ายมีหน่วยเดียวที่จำลองลักษณะของเซลล์ประสาทดังรูปที่ 6–35

ขั้นตอนการTest



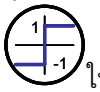
รูปที่ 6–35 เปอร์เซปตรอน

Linear Combination

$$x_0 w_0 + x_1 w_1 + x_2 w_2 + \ldots + x_n w_n$$

เพอร์เซปตรอนรับอินพุตเป็นเวกเตอร์จำนวนจริงแล้วคำนวณหาผลรวมเชิงเส้น (linear combination) แบบถ่วงน้ำหนักของอินพุต ($x_1, x_2, \ldots, x_n$) โดยที่ค่า $w_1, w_2, \ldots, w_n$ ในรูปเป็นค่าน้ำหนักของอินพุตและให้เอาต์พุต (o) เป็น 1 ถ้าผลรวมที่ได้มีค่าเกินค่าขีดแบ่ง (θ) และเป็น -1 ถ้าไม่เกิน ส่วน w_0 ในรูปเป็นค่าลบของค่าขีดแบ่งดังจะได้อธิบายต่อไป และ x_0 เป็นอินพุตเทียมกำหนดให้มีค่าเป็น 1 เสมอ

ฟังก์ชันกระตุ้น



ในรูปแสดง*ฟังก์ชันกระตุ้น (activation function)* ชนิดที่เรียกว่า*ฟังก์ชันสองขั้ว (bipolar function)* ซึ่งแสดงผลของเอาต์พุตเป็น 1 กับ -1 ฟังก์ชันกระตุ้นอื่นๆ ที่นิยมใช้ก็อย่างเช่น *ฟังก์ชันไบนารี (binary function)* ซึ่งแสดงผลของเอาต์พุตเป็น 1 กับ 0 และเขียนแทนด้วยรูป

เราสามารถแสดงเอาต์พุต (o) ในรูปของฟังก์ชันของอินพุต ($x_1, x_2, \ldots, x_n$) ได้ดังนี้

$$o(x_1, x_2, \ldots, x_n) = \begin{cases} 1 & if \;\; w_1x_1 + w_2x_2 + \cdots + w_nx_n > \theta \\ -1 & if \;\; w_1x_1 + w_2x_2 + \cdots + w_nx_n < \theta \end{cases} \tag{6.7}$$

เอาต์พุตเป็นฟังก์ชันของอินพุตในรูปของผลรวมเชิงเส้นแบบถ่วงน้ำหนัก น้ำหนักจะเป็นตัวกำหนดว่าในจำนวนอินพุตนั้น อินพุต (x_i) ตัวใดมีความสำคัญต่อการกำหนดค่าเอาต์พุต ตัวที่มีความสำคัญมากจะมีค่าสัมบูรณ์ของน้ำหนักมาก ส่วนตัวที่มีความสำคัญน้อยจะมีค่าใกล้ศูนย์ ในกรณีที่ผลรวมเท่ากับค่าขีดแบ่งค่าเอาต์พุตไม่นิยาม (จะเป็น 1 หรือ -1 ก็ได้)

จากฟังก์ชันในสูตรที่ (6.7) เราจัดรูปใหม่โดยย้าย θ ไปรวมกับผลรวมเชิงเส้นแล้วแทน $-\theta$ ด้วย w_0 เราจะได้ฟังก์ชันของเอาต์พุตดังด้านล่างนี้

$$o(x_1, x_2, \ldots, x_n) = \begin{cases} 1 & if \;\; w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n > 0 \\ -1 & if \;\; w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n < 0 \end{cases} \tag{6.8}$$

กำหนดให้ $g(\vec{x}) = \sum_{i=0}^{n} w_i x_i = \vec{w} \cdot \vec{x}$ โดยที่ $\vec{x}$ แทนเวกเตอร์อินพุต เราสามารถเขียนฟังก์ชันของเอาต์พุตได้ใหม่ดังนี้

$$o(x_1, x_2, \ldots, x_n) = \begin{cases} 1 & if \;\; g(\vec{x}) > 0 \\ -1 & if \;\; g(\vec{x}) < 0 \end{cases} \tag{6.9}$$

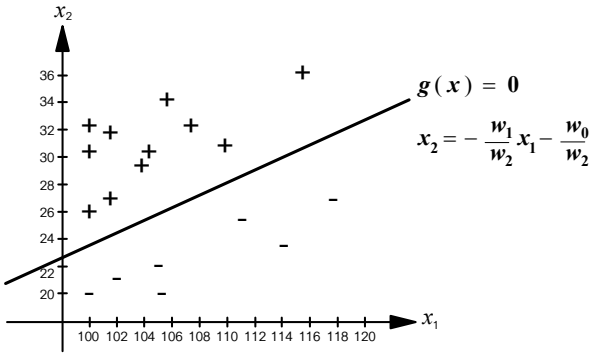
สมมติว่าเรามีอินพุตสองตัวคือ x_1 และ x_2 ซึ่งแสดงค่าส่วนสูงและน้ำหนักของเด็กนักเรียนประถมและหลังจากที่แพทย์ตรวจร่างกายของเด็กโดยละเอียดแล้วได้จำแนกนักเรียน

ออกเป็นสองกลุ่มคือเด็กอ้วนและเด็กไม่อ้วน เราให้เอาต์พุตเป็นค่าที่แสดงเด็กอ้วนแทนด้วย +1 กับไม่อ้วนแทนด้วย -1 ดังตารางที่ 6–16

ตารางที่ 6–16 ข้อมูลเด็กอ้วนและเด็กไม่อ้วน

เด็กคนที่	ส่วนสูง (ซม.)	น้ำหนัก (กก.)	อ้วน/ไม่อ้วน
1	100.0	20.0	-1
2	100.0	26.0	1
3	100.0	30.4	1
4	100.0	32.4	1
5	101.6	27.0	1
6	101.6	32.0	1
7	102.0	21.0	-1
8	103.6	29.6	1
9	104.4	30.4	1
10	104.9	22.0	-1
11	105.2	20.0	-1
12	105.6	34.4	1
13	107.2	32.4	1
14	109.9	34.9	1
15	111.0	25.4	-1
16	114.2	23.5	-1
17	115.5	36.3	1
18	117.8	26.9	-1

ในกรณีที่มีอินพุต 2 ตัว (ไม่รวม x_0) เราจะได้ $g(\vec{x}) = w_0 + w_1x_1 + w_2x_2$ ซึ่งถ้าเราให้ $g(\vec{x}) = 0$ จะได้ว่า $w_0 + w_1x_1 + w_2x_2 = 0$ ซึ่งแทนสมการเส้นตรงในระนาบสองมิติ x_1, x_2 สมการนี้มีจุดตัดแกนอยู่ที่ $-\frac{w_0}{w_2}$ และมีความชันเท่ากับ $-\frac{w_1}{w_2}$ เมื่อนำสมการนี้ไปวาดในระนาบสองมิติร่วมกับตัวอย่างสอนในตารางที่ 6–16 โดยกำหนดค่า w_0, w_1, w_2 ที่เหมาะสมจะได้ดังรูปที่ 6–36



รูปที่ 6–36 สมการเส้นตรงสร้างโดยเพอร์เซบตรอน

เครื่องหมาย + และ – ในรูปแทนตัวอย่างบวก (เด็กอ้วน) และตัวอย่างลบ (เด็กไม่อ้วน) ตามลำดับ ดังจะเห็นได้ในรูปว่าเส้นตรงนี้เมื่อกำหนดจุดตัดแกนและความชันที่เหมาะสมซึ่งกำหนดโดย w_0, w_1, w_2 เส้นตรงนี้จะแบ่งตัวอย่างออกเป็นสองกลุ่มซึ่งอยู่คนละด้านของเส้นตรง และเมื่อมีข้อมูลส่วนสูงและน้ำหนักของเด็กคนอื่นที่เราต้องการทำนายว่าจะเป็นเด็กอ้วนหรือไม่ ก็ใช้เส้นตรงนี้โดยดูว่าข้อมูลใหม่นี้อยู่ด้านใดของเส้นตรง ถ้าด้านบนก็ทำนายว่าเป็นเด็กอ้วน (+) ถ้าด้านล่างก็ทำนายว่าเด็กไม่อ้วน (–)

ระนาบตัดสินใจหลายมิติ

ตัวอย่างด้านบนแสดงกรณีของอินพุตในสองมิติ จะเห็นได้ว่าเพอร์เซปตรอนจะเป็นเส้นตรง ในกรณีที่อินพุตมากกว่าสองมิติเพอร์เซปตรอนจะเป็น*ระนาบตัดสินใจหลายมิติ (hyperplane decision surface)* ปัญหาการเรียนรู้เพอร์เซปตรอนก็คือการหาค่าเวกเตอร์น้ำหนัก ($\vec{w}$) ที่เหมาะสมในการจำแนกประเภทของข้อมูลสอนเพื่อให้เพอร์เซปตรอนแสดงเอาต์พุตได้ตรงกับค่าที่สอน *กฎการเรียนรู้เพอร์เซปตรอน (perceptron learning rule)* ใช้สำหรับสอนเพอร์เซปตรอนโดยจะหาค่าเวกเตอร์น้ำหนักดังแสดงในตารางที่ 6–17

อัลกอริทึมเริ่มต้นจากสุ่มค่าเวกเตอร์น้ำหนัก ซึ่งโดยมากค่าที่สุ่มมานี้จะไม่ได้ระนาบหลายมิติที่แบ่งตัวอย่างได้ถูกต้องทุกตัวดังนั้นจึงต้องมีการแก้ไขน้ำหนักโดยเทียบเพอร์-เซปตรอนกับตัวอย่างที่สอน หมายถึงว่าเมื่อเราป้อนตัวอย่างสอนเข้าไปในเพอร์เซปตรอนเราจะคำนวณค่าเอาต์พุตได้ นำค่าเอาต์พุตที่คำนวณได้โดยเพอร์เซปตรอนเทียบกับเอาต์พุตเป้าหมาย ถ้าตรงกันแสดงว่าจำแนกตัวอย่างได้ถูกต้อง ไม่ต้องปรับน้ำหนักสำหรับตัวอย่างนั้น แต่ถ้าไม่ตรงกันก็จะทำการปรับน้ำหนักตามสมการในอัลกอริทึม ส่วนอัตราการเรียนรู้เป็นตัวเลขบวกจำนวนน้อยๆ เช่น 0.01, 0.005 เป็นต้น อัตราการเรียนรู้นี้จะส่งผลต่อการลู่เข้าของเพอร์เซปตรอน ถ้าอัตราการเรียนรู้มีค่ามากเพอร์เซปตรอนก็จะเรียนรู้ได้เร็วแต่ก็อาจเรียนรู้ไม่สำเร็จเนื่องจากการปรับค่ามีความหยาบเกินไป อัตราการเรียนรู้ที่มีค่าน้อยก็จะทำให้การปรับน้ำหนักทำได้อย่างละเอียดแต่ก็อาจเสียเวลาในการเรียนรู้นาน

ตารางที่ 6–17 อัลกอริทึมกฎการเรียนรู้เพอร์เซปตรอน

Algorithm: Perceptron-Learning-Rule

0 ง่ายๆ

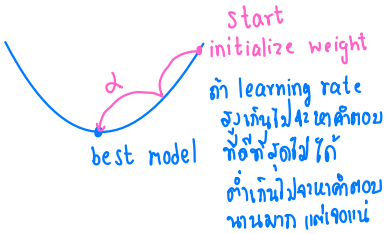
1. Initialize weights w_i of the perceptron.
2. **UNTIL** the termination condition is met **DO**
   2.1 **FOR EACH** training example **DO**
   - Input the example and compute the output.
   - Change the weights if the output from the perceptron is not equal to the target output using the following rule.

พจุนม่ พเก่า ค่าเปลี่ยนแปลง พ

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i \leftarrow \alpha(t-o)x_i$$

where t, o and α are the target output, the output from the perceptron and the learning rate, respectively.



α learning rate

x_i input

การปรับน้ำหนักตามกฎการเรียนรู้เพอร์เซปตรอนโดยใช้อัตราการเรียนรู้ที่มีค่าน้อยเพียงพอ จะได้ระนาบหลายมิติที่จะลู่เข้าสู่ระนาบหนึ่งที่สามารถแบ่งข้อมูลออกเป็นสองส่วน (ในกรณีที่ข้อมูลสามารถแบ่งได้) เพื่ออธิบายผลที่เกิดจากการปรับค่าน้ำหนัก เราจะลองพิจารณาพฤติกรรมของกฎการเรียนรู้นี้ดูว่าทำไมการปรับน้ำหนักเช่นนี้จึงลู่เข้าสู่ระนาบที่แบ่งข้อมูลได้อย่างถูกต้อง

- พิจารณากรณีแรกที่เพอร์เซปตรอนแยกตัวอย่างสอนตัวหนึ่งที่รับเข้ามาได้ถูกต้อง กรณีนี้จะพบว่า (t-o) จะมีค่าเป็น 0 ดังนั้น Δw_i ไม่เปลี่ยนแปลงเพราะ $\Delta w_i = \alpha(t\text{-}o)x_i$
- พิจารณาในกรณีที่เพอร์เซปตรอนให้เอาต์พุตเป็น –1 แต่เอาต์พุตเป้าหมายหรือค่าที่แท้จริงเท่ากับ 1 ในกรณีนี้หมายความว่าค่าที่เราต้องการคือ 1 แต่ค่าน้ำหนักไม่เหมาะสม ดังนั้นเพื่อที่จะทำให้เพอร์เซปตรอนให้เอาต์พุตเป็น 1 น้ำหนักต้องถูกปรับให้สามารถเพิ่มค่าของ $\vec{w}\cdot\vec{x}$ ในกรณีนี้หมายความว่าผลรวมเชิงเส้นน้อยเกินไปและน้อยกว่า 0 จึงได้เอาต์พุตเป็น -1 ดังนั้นสิ่งที่เราต้องการคือการเพิ่มค่าผลรวมเชิงเส้นเพราะถ้าเราเพิ่มค่าได้เรื่อยๆ จนมากกว่า 0 เพอร์เซปตรอนจะให้เอาต์พุตเป็น 1 ซึ่งตรงกับที่เราต้องการ พิจารณาดูดังต่อไปนี้ว่าการปรับค่าโดยกฎเรียนรู้ทำให้ผลรวมเชิงเส้นเพิ่มขึ้นได้อย่างไร กรณีนี้เราจะได้ว่า (t-o) เท่ากับ (1-(-1)) มีค่าเป็น 2 และลองพิจารณาค่าของอินพุต x_i แยกกรณีดังนี้

- ถ้า x_i > 0 จะได้ว่า Δw_i มากกว่า 0 เพราะว่า $\Delta w_i \leftarrow \alpha(t-o)x_i$ และ α มากกว่า 0, (t-o) = 2 และ x_i > 0 จากสมการการปรับน้ำหนัก $w_i \leftarrow w_i + \Delta w_i$ เมื่อ Δw_i มากกว่า 0 จะทำให้ w_i มีค่าเพิ่มขึ้นและ $\sum w_i x_i$ ก็จะมีค่าเพิ่มขึ้น เมื่อผลรวมมีค่ามากขึ้นแสดงว่าการปรับไปในทิศทางที่ถูกต้องคือเมื่อปรับไปจนกระทั่งได้ผลรวมมากกว่า 0 จะทำให้เพอร์เซปตรอนเอาต์พุตได้ถูกต้องยิ่งขึ้น
- ถ้า x_i < 0 เราจะได้ว่า $\alpha(t-o)x_i$ จะมีค่าน้อยกว่า 0 แสดงว่า w_i ตัวที่คูณกับ x_i ที่น้อยกว่า 0 จะลดลงทำให้ $\sum w_i x_i$ เพิ่มขึ้นเหมือนเดิม เพราะ x_i เป็นค่าลบและ w_i มีค่าลดลง ในที่สุดก็จะทำให้เพอร์เซปตรอนให้เอาต์พุตได้ถูกต้องยิ่งขึ้น

- ในกรณีที่เพอร์เซปตรอนให้เอาต์พุตเป็น 1 แต่เอาต์พุตเป้าหมายหรือค่าที่แท้จริงเท่ากับ -1 จะได้ว่า w_i ของ x_i ที่เป็นค่าบวกจะลดลง ส่วน w_i ของ x_i ที่เป็นค่าลบจะเพิ่มขึ้นและทำให้การปรับเป็นไปในทิศทางที่ถูกต้องเช่นเดียวกับในกรณีแรก

6.7.2 ตัวอย่างการเรียนฟังก์ชัน AND และ XOR ด้วยกฎเรียนรู้เพอร์เซปตรอน

พิจารณาตัวอย่างการเรียนรู้ของเพอร์เซปตรอนโดยจะให้เรียนรู้ฟังก์ชัน 2 ฟังก์ชัน ฟังก์ชันแรกคือฟังก์ชัน AND แสดงในตารางที่ 6–18 ในกรณีนี้เราใช้ฟังก์ชันไบนารีเป็นฟังก์ชันกระตุ้น

ตารางที่ 6–18 ฟังก์ชัน AND(x1,x2)

x_1	x_2	เอาต์พุตเป้าหมาย
0	0	0
0	1	0
1	0	0
1	1	1

ฟังก์ชัน AND ตามตารางด้านบนนี้จะให้ค่าที่เป็นจริงก็ต่อเมื่อ x1 และ x2 เป็นจริงทั้งคู่ (ดูที่สดมภ์เอาต์พุตเป้าหมาย) ผลการใช้กฎการเรียนรู้เพอร์เซปตรอนกับฟังก์ชัน AND แสดงในตารางที่ 6–19

Model 4 แบบ Naiv Bey – ง่ายสุด

Final
ใช้ง่าย Model ไหนแม่นสุด
ใน Data ที่อาจให้สอบ

$x_0 = 1$, x_1, x_2, w_0, w_1, w_2, Σ

ตารางที่ 6–19 ผลการเรียนรู้ฟังก์ชัน AND โดยกฎการเรียนรู้เพอร์เซปตรอน

Perceptron Learning Example - Function AND											
						Output ของ model				t – O	
		Bias Input x0=+1					Alpha =	0.5			
Input	Input				Net Sum	Target	Actual	Alpha*	Weight Values		
x1	x2	1.0*w0	x1*w1	x2*w2	Input	Output	Output	Error	w0	w1	w2
									0.1	0.1	0.1
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	0.10	0.10
0	1	-0.40	0.00	0.10	-0.30	0	0	0.00	-0.40	0.10	0.10
1	0	-0.40	0.10	0.00	-0.30	0	0	0.00	-0.40	0.10	0.10
1	1	-0.40	0.10	0.10	-0.20	1	0	0.50	0.10	0.60	0.60
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	0.60	0.60
0	1	-0.40	0.00	0.60	0.20	0	1	-0.50	-0.90	0.60	0.10
1	0	-0.90	0.60	0.00	-0.30	0	0	0.00	-0.90	0.60	0.10
1	1	-0.90	0.60	0.10	-0.20	1	0	0.50	-0.40	1.10	0.60
0	0	-0.40	0.00	0.00	-0.40	0	0	0.00	-0.40	1.10	0.60
0	1	-0.40	0.00	0.60	0.20	0	1	-0.50	-0.90	1.10	0.10
1	0	-0.90	1.10	0.00	0.20	0	1	-0.50	-1.40	0.60	0.10
1	1	-1.40	0.60	0.10	-0.70	1	0	0.50	-0.90	1.10	0.60
0	0	-0.90	0.00	0.00	-0.90	0	0	0.00	-0.90	1.10	0.60
0	1	-0.90	0.00	0.60	-0.30	0	0	0.00	-0.90	1.10	0.60
1	0	-0.90	1.10	0.00	0.20	0	1	-0.50	-1.40	0.60	0.60
1	1	-1.40	0.60	0.60	-0.20	1	0	0.50	-0.90	1.10	1.10
0	0	-0.90	0.00	0.00	-0.90	0	0	0.00	-0.90	1.10	1.10
0	1	-0.90	0.00	1.10	0.20	0	1	-0.50	-1.40	1.10	0.60
1	0	-1.40	1.10	0.00	-0.30	0	0	0.00	-1.40	1.10	0.60
1	1	-1.40	1.10	0.60	0.30	1	1	0.00	-1.40	1.10	0.60
0	0	-1.40	0.00	0.00	-1.40	0	0	0.00	-1.40	1.10	0.60
0	1	-1.40	0.00	0.60	-0.80	0	0	0.00	-1.40	1.10	0.60
1	0	-1.40	1.10	0.00	-0.30	0	0	0.00	-1.40	1.10	0.60
1	1	-1.40	1.10	0.60	0.30	1	1	0.00	-1.40	1.10	0.60

Error = target – output

$w_{0\text{ใหม่}} = w_0 + \alpha(t-o)x_0$
$= 0.1 + (-0.5)(1)$
$= 0.4$

0.5(-1)

ตอบ ✓ ไม่ถูกปรับ

ครบ 1 รอบ Epoch

ต้องมี 10 ว

รัก ข้อมูลเยอะ
set epoch ให้พอตอบได้ส่วนมาก
ถ้าถูกหมดอาจ fit กับ noise ได้

epoch ที่ 6 ✓ หมด

ขั้นตอนแรกเริ่มจากการสุ่มค่า w_0 จนถึง w_2 ในที่นี้กำหนดให้เป็น 0.1 ทั้งสามตัว จากนั้นก็เริ่มป้อนตัวอย่างเข้าไป (ทีละแถว) ตัวอย่างแรกได้ผลรวมเชิงเส้น (Net Sum) เป็น 0.10 ซึ่งมากกว่า 0 ดังนั้นเปอร์เซปตรอนจะให้เอาต์พุตจริง (Actual Output) ออกมาเป็น 1 ซึ่งผิด เพราะเอาต์พุตเป้าหมาย (Target Output) จะต้องได้เป็น 0 ทำให้อัตราการเรียนรู้คูณค่าผิดพลาด (Alpha x Error) ได้ –0.50 หลังจากนี้ก็นำไปปรับน้ำหนักตาม $w_i \leftarrow w_i + \Delta w_i$ และ $\Delta w_i \leftarrow \alpha(t-o)x_i$ ดังนั้นจะได้เป็น $w_0 \leftarrow w_0 + \alpha(t-o)x_0$ = w_0 + 0.50(-1) x 1 = 0.10 + (-0.5) = -0.4 ต่อไปก็ปรับค่า w_1 ในทำนองเดียวกัน $w_1 \leftarrow w_1 + \alpha(t-o)x_1$ = w_1 + 0.50(-1) x 0 ดังนั้น w_1 จะเท่ากับ 0.10 คือไม่เปลี่ยนแปลง เช่นเดียวกับ w_2 ที่ไม่เปลี่ยนแปลง จะเห็นได้ว่าแม้มีค่าผิดพลาดแต่ไม่มีการปรับค่า w_1 และ w_2 เนื่องจากอินพุตที่ใส่เข้าไปเป็น 0 ทำ

ให้ผลคูณเป็น 0 จึงไม่ได้ปรับ และเป็นข้อเสียของฟังก์ชันกระตุ้นแบบไบนารีซึ่งถ้าผลออกมาเป็น 0 จะไม่มีการปรับค่าให้ (ถ้าเราเปลี่ยน 0 เป็น –1 การปรับค่าจะดีขึ้น w_i จะถูกปรับทันทีตั้งแต่รอบแรก)

ตัวอย่างที่สองจนถึงตัวอย่างที่สี่ก็ทำเช่นเดียวกัน และเมื่อทำครบ 1 รอบการสอน (epoch) แล้วจะต้องทำการสอนซ้ำด้วยข้อมูลชุดเดิม นี่คือวิธีการสอนของข่ายงานประสาทเทียมซึ่งต่างจากวิธีอื่นๆ ที่ต้องใช้ข้อมูลชุดเดิมสอนซ้ำไปจนกระทั่งค่าผิดพลาดลดลงจนถึงจุดที่เราต้องการ ในที่นี้คือ 0 เนื่องจากเราต้องการให้มีการแบ่งข้อมูลอย่างเด็ดขาด และสมการเส้นตรงที่ได้จะมีค่า w_0 = -1.40, w_1 = 1.10 และ w_2 = 0.60

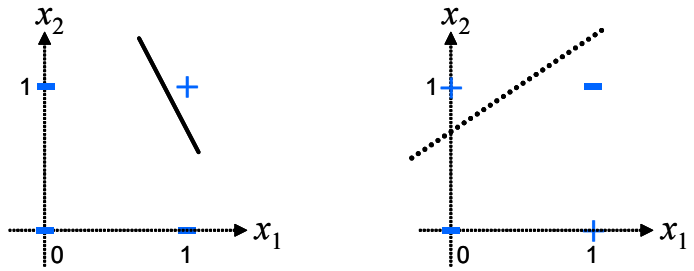
ฟังก์ชันที่สองที่จะทดลองเรียนรู้ด้วยกฎการเรียนรู้เพอร์เซปตรอนคือฟังก์ชัน XOR แสดงในตารางที่ 6–20

ตารางที่ 6–20 ฟังก์ชัน XOR(x1,x2)

x_1	x_2	เอาต์พุตเป้าหมาย
0	0	0
0	1	1
1	0	1
1	1	0

ฟังก์ชัน XOR ตามตารางด้านบนนี้จะให้ค่าที่เป็นจริงก็ต่อเมื่อ x_1 หรือ x_2 ตัวใดตัวหนึ่งเพียงตัวเดียวเป็นจริง (ดูที่สดมภ์เอาต์พุตเป้าหมาย) ผลการใช้กฎการเรียนรู้เพอร์เซปตรอนกับฟังก์ชัน XOR แสดงในตารางที่ 6–21

ในกรณีของฟังก์ชัน XOR นี้พบว่าค่าผิดพลาดไม่ลดลง และค่าน้ำหนักจะแกว่งไปมาโดยไม่ลู่เข้าแม้ว่าจะสอนต่อจากนี้ไปอีกกี่รอบการสอนก็ตาม จึงสรุปว่าฟังก์ชัน XOR เรียนไม่สำเร็จด้วยเพอร์เซปตรอน เมื่อเรานำฟังก์ชัน AND และ XOR ไปวาดกราฟในสองมิติจะได้กราฟดังรูปที่ 6–37



(ก) ฟังก์ชันแยกเชิงเส้นได้ (AND) (ข) ฟังก์ชันแยกเชิงเส้นไม่ได้ (XOR)

รูปที่ 6–37 ฟังก์ชันแยกเชิงเส้นได้และไม่ได้

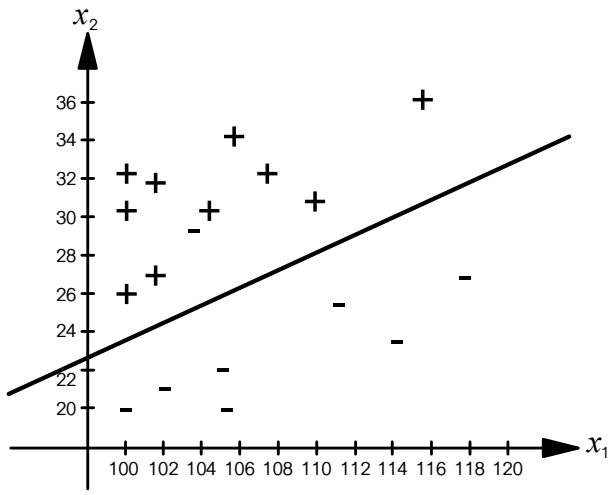
ตารางที่ 6–21 ผลการเรียนรู้ฟังก์ชัน XOR โดยกฎการเรียนรู้เพอร์เซปตรอน

Perceptron Learning Example XOR											
		Bias Input X0=+1					Alpha =	0.5			
Input	Input				Net Sum	Target	Actual	Alpha*	Weight Values		
x1	x2	1.0*w0	x1*w1	x2*w2	Input	Output	Output	Error	w0	w1	w2
									0.1	0.1	0.1
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	0.10	0.10
0	1	-0.40	0.00	0.10	-0.30	1	0	0.50	0.10	0.10	0.60
1	0	0.10	0.10	0.00	0.20	1	1	0.00	0.10	0.10	0.60
1	1	0.10	0.10	0.60	0.80	0	1	-0.50	-0.40	-0.40	0.10
0	0	-0.40	0.00	0.00	-0.40	0	0	0.00	-0.40	-0.40	0.10
0	1	-0.40	0.00	0.10	-0.30	1	0	0.50	0.10	-0.40	0.60
1	0	0.10	-0.40	0.00	-0.30	1	0	0.50	0.60	0.10	0.60
1	1	0.60	0.10	0.60	1.30	0	1	-0.50	0.10	-0.40	0.10
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	-0.40	0.10
0	1	-0.40	0.00	0.10	-0.30	1	0	0.50	0.10	-0.40	0.60
1	0	0.10	-0.40	0.00	-0.30	1	0	0.50	0.60	0.10	0.60
1	1	0.60	0.10	0.60	1.30	0	1	-0.50	0.10	-0.40	0.10
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	-0.40	0.10
0	1	-0.40	0.00	0.10	-0.30	1	0	0.50	0.10	-0.40	0.60
1	0	0.10	-0.40	0.00	-0.30	1	0	0.50	0.60	0.10	0.60
1	1	0.60	0.10	0.60	1.30	0	1	-0.50	0.10	-0.40	0.10
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	-0.40	0.10
0	1	-0.40	0.00	0.10	-0.30	1	0	0.50	0.10	-0.40	0.60
1	0	0.10	-0.40	0.00	-0.30	1	0	0.50	0.60	0.10	0.60
1	1	0.60	0.10	0.60	1.30	0	1	-0.50	0.10	-0.40	0.10

จากรูปจะเห็นได้ว่าฟังก์ชัน AND เป็นฟังก์ชันที่แยก (ระหว่างตัวอย่างบวกกับตัวอย่างลบ) ได้ด้วยเส้นตรง ส่วนฟังก์ชัน XOR เราไม่สามารถหาเส้นตรงที่มาแบ่งตัวอย่างบวกและลบออกจากกัน (ไม่สามารถลากเส้นตรงให้ตัวอย่างบวกและลบให้อยู่คนละด้านของเส้น) ตัวอย่างการเรียนรู้ฟังก์ชัน XOR ข้างต้นได้แสดงให้เห็นว่า เพอร์เซปตรอนเรียนรู้บางฟังก์ชันไม่ได้ ฟังก์ชันเหล่านี้เรียกว่า *ฟังก์ชันแยกเชิงเส้นไม่ได้ (linearly non-separable function)* ส่วนฟังก์ชันที่แยกได้เรียกว่า *ฟังก์ชันแยกเชิงเส้นได้ (linearly separable function)* ซึ่งเป็นข้อจำกัดของเพอร์เซปตรอน เมื่อเราย้อนกลับไปดูตารางที่ 6–21 จะพบว่านอกจากการเรียนรู้ฟังก์ชัน XOR ไม่สำเร็จแล้ว การเรียนรู้ก็จะไม่ลู่เข้าสู่เส้นตรงใดเส้นตรงหนึ่งอีกด้วย ดังจะเห็นได้จากการที่เวกเตอร์น้ำหนักจะแกว่งไปมา การไม่ลู่เข้าก่อให้เกิดปัญหาในการเรียนรู้เพราะเราจะไม่รู้ว่าเมื่อไรจะหยุดอัลกอริทึม พิจารณาตัวอย่างใน

ฟังก์ชันแยกเชิงเส้นไม่ได้

รูปที่ 6–38 ซึ่งมีตัวอย่างสอนเหมือนกับในรูปที่ 6–36 ยกเว้นว่าตัวอย่างตัวที่แปดในตารางที่ 6–16 มีการบันทึกค่าของประเภทผิดจาก 1 เป็น -1 ทำให้เกิดตัวอย่างลบปะปนไปในกลุ่มของตัวอย่างบวกดังแสดงในรูป ในกรณีเช่นนี้เส้นตรงที่ดีที่สุดก็ยังคงเป็นเส้นตรงเดิมเหมือนกับในรูปที่ 6–36 แต่ว่ากฎการเรียนรู้เพอร์เซปตรอนจะไม่ให้คำตอบเป็นเส้นตรงนี้ เนื่องจากอัลกอริทึมไม่ลู่เข้าสู่เส้นตรงเดียว แต่จะแกว่งไปมา

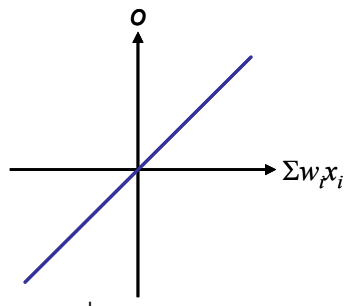


รูปที่ 6–38 เส้นตรงที่ให้ค่าผิดพลาดน้อยสุด

กฎเดลต้า (delta rule) เป็นกฎการเรียนรู้สำหรับหาค่าเวกเตอร์น้ำหนักของเพอร์เซปตรอนอีกกฎหนึ่งและมีข้อดีที่การเรียนรู้จะลู่เข้าสู่ระนาบหลายมิติที่ให้ค่าผิดพลาดน้อยสุด แม้ว่าตัวอย่างจะเป็นฟังก์ชันแบบแยกเชิงเส้นไม่ได้ กฎนี้ใช้หลักการของ*การเคลื่อนลงตามความชัน (gradient descent)* เพื่อหาคำตอบจากปริภูมิของเวกเตอร์น้ำหนักที่เป็นไปได้ ซึ่งกฎนี้เป็นพื้นฐานของอัลกอริทึมการแพร่กระจายย้อนกลับ (back-propagation) ดังจะกล่าวต่อไป

กฎเดลต้านี้จะหาเวกเตอร์น้ำหนักที่ให้ค่าผิดพลาดของตัวอย่างสอนน้อยสุดโดยใช้การหาอนุพันธ์ทางคณิตศาสตร์ ซึ่งในการหาค่าน้อยสุดด้วยอนุพันธ์นั้นจำเป็นต้องใช้ฟังก์ชันกระตุ้นที่หาอนุพันธ์ได้ ฟังก์ชันที่เราเคยใช้ก่อนหน้านี้เช่นฟังก์ชันสองขั้วและฟังก์ชันไบนารีเป็นฟังก์ชันที่หาอนุพันธ์ไม่ได้ในบางจุด ดังนั้นในกฎเดลต้านี้เราจะใช้ฟังก์ชันกระตุ้นแบบ*ฟังก์ชันเชิงเส้น (linear function)* ดังแสดงในรูปที่ 6–39 ซึ่งค่าเอาต์พุต (o) แสดงโดย $o(\vec{x}) = \vec{w} \cdot \vec{x} = \sum w_i x_i$ กล่าวคือเอาต์พุตจะเท่ากับผลรวมเชิงเส้น แม้ว่าตัวอย่างสอนจะแบ่งเป็นสองกลุ่ม (เช่น 1 กับ -1) ก็ไม่เกิดปัญหาเมื่อเราใช้ฟังก์ชันเชิงเส้นโดยจะทำนาย

ประเภทของตัวอย่างได้โดยดูที่เครื่องหมาย เช่นถ้าฟังก์ชันเชิงเส้นให้เอาต์พุตเป็น -0.21 ก็ให้ทำนายประเภทเป็น -1 เป็นต้น



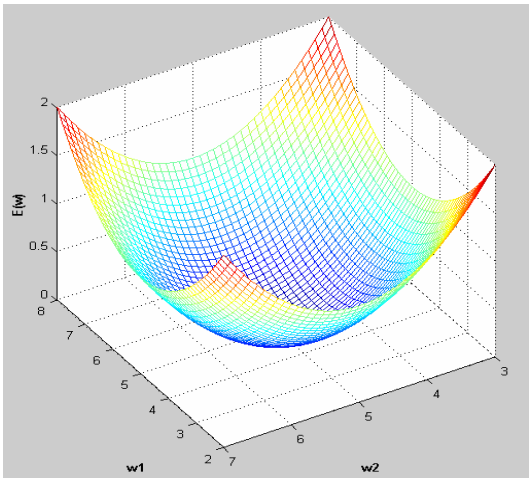
รูปที่ 6–39 ฟังก์ชันเชิงเส้น

ดังที่กล่าวข้างต้น กฎเดลต้าจะหาค่าเวกเตอร์น้ำหนักที่ให้ค่าผิดพลาดต่ำสุด ดังนั้นเรานิยามฟังก์ชันค่าผิดพลาดการสอน (training error function) $E(\vec{w})$ ดังนี้

$$E(\vec{w}) = \frac{1}{2}\sum_{d \in D}(t_d - o_d)^2 \tag{6.10}$$

โดยที่ D เป็นเซตของตัวอย่างสอน t_d เป็นเอาต์พุตเป้าหมายของตัวอย่าง d และ o_d เป็นเอาต์พุตของเพอร์เซปตรอนสำหรับตัวอย่าง d

ฟังก์ชันค่าผิดพลาดการสอน $E(\vec{w})$ เป็นฟังก์ชันของ $\vec{w}$ จะมีค่า $\vec{w}$ บางตัวที่ทำให้ฟังก์ชันมีค่าต่ำสุด และพบว่าจะมี $\vec{w}$ เช่นนั้นแค่ตัวเดียวเพราะ $E(\vec{w})$ เป็นฟังก์ชันพาราโบล่าของ $\vec{w}$ ในกรณีที่ $\vec{w}$ ประกอบด้วยน้ำหนัก 2 ค่าคือ w_1 และ w_2 เราจะได้ฟังก์ชันดังรูปที่ 6–40



รูปที่ 6–40 ฟังก์ชันค่าผิดพลาดการสอน $E(\vec{w})$

คุณสมบัติของฟังก์ชันพาราโบล่าคือจะมีค่าต่ำสุดเพียงค่าเดียว ในการหาค่าต่ำสุดเราสามารถทำได้โดยกำหนดจุด (w_1,w_2) เริ่มต้น สมมติว่าเป็น (w_{10},w_{20}) จากนั้นหาเวกเตอร์สัมผัสพาราโบล่า ณ ตำแหน่ง $E(w_{10},w_{20})$ แล้วเราจะวิ่งลงตามความชันของเวกเตอร์ที่สัมผัสกับผิวค่าผิดพลาด (error surface) ถ้าชันมากก็ปรับค่าเวกเตอร์น้ำหนักมาก ถ้าชันน้อยก็ปรับค่าน้อยจนกระทั่งมาถึงจุดต่ำสุด ซึ่ง ณ จุดนี้ความชันจะเท่ากับศูนย์และไม่ต้องปรับค่าเวกเตอร์น้ำหนักอีกต่อไป ดังนั้นการใช้หลักการนี้ต้องการการหาอนุพันธ์ของผิวค่าผิดพลาด ซึ่งจะได้เป็นความชันของผิวสัมผัสกับผิวค่าผิดพลาด $E(\vec{w})$ นี้ (เขียนแทนด้วย $\nabla E(\vec{w})$) ดังแสดงต่อไปนี้

$$\nabla E(\vec{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n} \right] \tag{6.11}$$

เนื่องจากเวกเตอร์สัมผัสนี้มีทิศในแนวขึ้น แต่เราต้องการวิ่งลงดังนั้นเวกเตอร์ในแนวลงจึงเป็น $-\nabla E(\vec{w})$ เราจะได้ว่ากฎการปรับค่าเวกเตอร์น้ำหนักเป็น $\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$ โดยที่ $\Delta\vec{w} = -\eta\nabla E(\vec{w})$ และ η คืออัตราการเรียนรู้เป็นค่าคงที่ตัวเลขบวก กฎเดลต้านี้สามารถเขียนให้อยู่ในรูปของสมาชิกแต่ละตัวของเวกเตอร์น้ำหนักได้เป็น $w_i \leftarrow w_i + \Delta w_i$ โดยที่ $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$

$\frac{\partial E}{\partial w_i}$ สามารถคำนวณได้ดังต่อไปนี้

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_{d \in D} \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_{d \in D} 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
&= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\
\frac{\partial E}{\partial w_i} &= \sum_{d \in D} (t_d - o_d)(-x_{id})
\end{aligned}$$

โดยที่ $-x_{id}$ คือสมาชิก x_i ของตัวอย่าง d

$$\therefore \Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id} \tag{6.12}$$

อัลกอริทึมของกฎเดลต้าเป็นดังตารางที่ 6–22 ต่อไปนี้

ตารางที่ 6–22 อัลกอริทึมกฎเดลต้า

Algorithm: Delta-Rule(training-examples,η)

Each training example is a pair $<\vec{x},t>$, where $\vec{x}$ is the vector of input values, and t is the target output value. η is the learing rate.

1. Initialize each w_i to some small random value.
2. **UNTIL** the termination condition is met **DO**
   2.1 Initialize each Δw_i to zero.
   2.2 **FOR EACH** $<\vec{x},t>$ in training-examples **DO**
      - Input the instance $\vec{x}$ to the unit and compute the output o.
      - **FOR EACH** linear unit weight w_i **DO**

$$\Delta w_i \leftarrow \Delta w_i + \eta(t-o)x_i$$

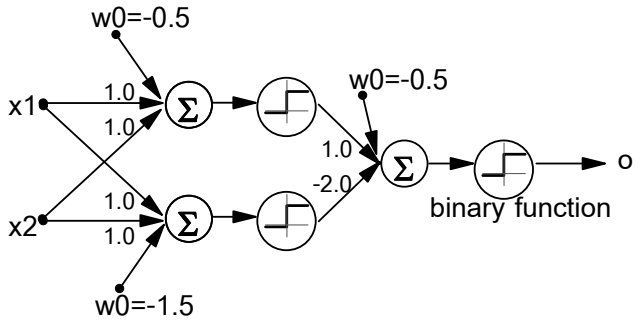
   2.3 **FOR EACH** linear weight w_i **DO**

$$w_i \leftarrow w_i + \Delta w_i$$

อัลกอริทึมกฎเดลต้าข้างต้นนี้จะหาค่าเวกเตอร์น้ำหนักที่ให้ค่าความผิดพลาดน้อยสุด ซึ่งมีข้อดีที่อัลกอริทึมจะลู่เข้า อย่างไรก็ดีฟังก์ชันแยกเชิงเส้นไม่ได้ที่เรียนรู้ไม่ได้ด้วยกฎเรียนรู้เพอร์เซปตรอนก็ไม่สามารถแยกได้อย่างถูกต้องสมบูรณ์ด้วยกฎเดลต้าเช่นกัน ในหัวข้อต่อไปจะกล่าวถึงข่ายงานหลายชั้นที่สามารถแยกฟังก์ชันประเภทนี้ได้

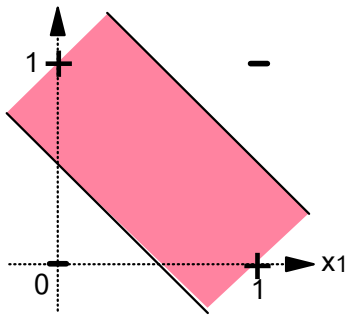
6.7.3 ข่ายงานหลายชั้นและการแพร่กระจายย้อนกลับ

จากข้างต้นจะเห็นว่าเพอร์เซปตรอนสามารถเรียนรู้ฟังก์ชันแยกได้เชิงเส้นเท่านั้น ในส่วนนี้จะอธิบายการนำเพอร์เซปตรอนหลายๆ ตัวมาเชื่อมต่อกัน เพื่อสร้างเป็นข่ายงานประสาทหลายชั้น (multilayer neural network) ที่สามารถแสดงผิวตัดสินใจไม่เชิงเส้น (non-linear decision surface) เพื่อให้เห็นถึงประสิทธิภาพของข่ายงานหลายชั้น จะยกตัวอย่างการต่อเพอร์เซปตรอน 3 ตัวเข้าด้วยกันเพื่อเรียนรู้ฟังก์ชัน XOR ดังแสดงในรูปที่ 6–41



รูปที่ 6–41 ข่ายงานหลายชั้นสามารถเรียนรู้ฟังก์ชัน XOR

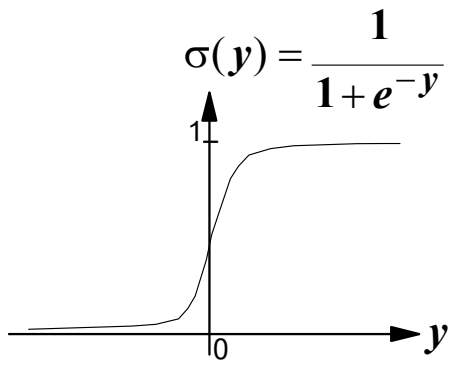
รูปที่ 6–41 แสดงการเชื่อมต่อเพอร์เซปตรอน 3 ตัวเข้าด้วยกัน เพอร์เซปตรอนสองตัวแรกรับอินพุตโดยตรงส่วนเพอร์เซปตรอนตัวที่สามรับอินพุตจากเอาต์พุตของเพอร์เซปตรอนสองตัวแรก จะเห็นได้ว่าเพอร์เซปตรอนตัวแรกที่อยู่ด้านซ้ายบนของรูปนั้นแทนฟังก์ชันเชิงเส้น $x_1 + x_2$ = 0.5 ส่วนเพอร์เซปตรอนตัวที่สองที่อยู่ด้านซ้ายล่างของรูปนั้นแทนฟังก์ชันเชิงเส้น $x_1 + x_2$ = 1.5 ฟังก์ชันเชิงเส้นทั้งสองมีความชันเท่ากันเท่ากับ -1 แต่มีจุดตัดแกนต่างกันดังแสดงในรูปที่ 6–42 ส่วนเพอร์เซปตรอนตัวที่สามทำหน้าที่รวมผลลัพธ์จากเพอร์เซปตรอนสองตัวแรก และโดยการกำหนดเวกเตอร์น้ำหนักที่เหมาะสมของเพอร์เซปตรอนตัวที่สามทำให้ได้ผิวตัดสินใจที่อยู่ระหว่างเส้นตรงทั้งสองเป็นตัวอย่างบวกและที่อยู่ด้านนอกเป็นตัวอย่างลบ



x1	x2	x1 XOR x2
0	0	0
0	1	1
1	0	1
1	1	0

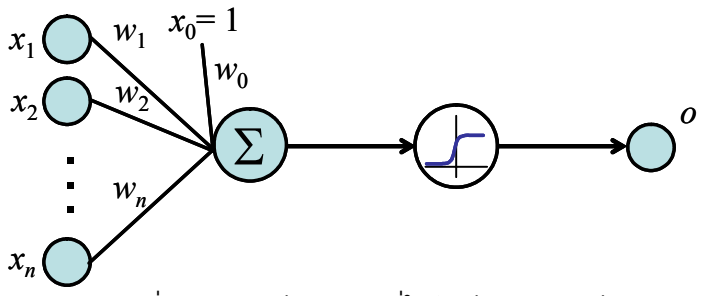
รูปที่ 6–42 ผิวตัดสินใจของข่ายงานในรูปที่ 6–41

ในการเชื่อมต่อครั้งนี้ใช้ฟังก์ชันกระตุ้นแบบไบนารีเพื่อให้ง่ายต่อการทำความเข้าใจ แต่การคำนวณหากฎเรียนรู้สำหรับข่ายงานหลายชั้นต้องใช้ฟังก์ชันกระตุ้นที่หาอนุพันธ์ได้ ดังนั้นเราจะไม่ใช้ฟังก์ชันไบนารีกับข่ายงานหลายชั้น แต่จะใช้ฟังก์ชันเชิงเส้นหรืออาจใช้ฟังก์ชันซิกมอยด์ (sigmoid function) ดังแสดงในรูปที่ 6–43



รูปที่ 6–43 ฟังก์ชันซิกมอยด์

เพอร์เซปตรอนที่ใช้ฟังก์ชันกระตุ้นเป็นฟังก์ชันซิกมอยด์แสดงในรูปที่ 6–44

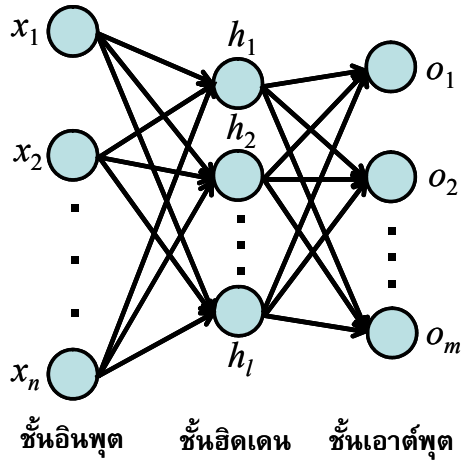


รูปที่ 6–44 เพอร์เซปตรอนที่ใช้ฟังก์ชันซิกมอยด์

คุณสมบัติหนึ่งของฟังก์ชันซิกมอยด์ก็คือสามารถแสดงอนุพันธ์ของฟังก์ชันในรูปของเอาต์พุตได้อย่างง่าย กล่าวคือ

$$\frac{d\sigma(y)}{dy} = \sigma(y)(1-\sigma(y)) \tag{6.13}$$

อัลกอริทึมการแพร่กระจายย้อนกลับ (backpropagation algorithm) [Rumelhart & McClelland, 1986] เรียนรู้ค่าเวกเตอร์น้ำหนักสำหรับข่ายงานป้อนไปหน้าแบบหลายชั้น (multilayer feedforward network) โดยใช้การเคลื่อนลงตามความชันเพื่อหาค่าต่ำสุดของค่าผิดพลาดระหว่างเอาต์พุตของข่ายงานกับเอาต์พุตเป้าหมาย ตัวอย่างของข่ายงานป้อนไปหน้าแบบหลายชั้นแสดงในรูปที่ 6–45



รูปที่ 6–45 ตัวอย่างข่ายงานป้อนไปหน้าแบบหลายชั้น

ตัวอย่างในรูปด้านบนแสดงข่ายงานป้อนไปหน้าแบบหลายชั้นซึ่งประกอบด้วยชั้นอินพุต ชั้นฮิดเดนหรือชั้นซ่อน และชั้นเอาต์พุต ในรูปแสดงชั้นฮิดเดนเพียงชั้นเดียวแต่อาจมีมากกว่าหนึ่งชั้นก็ได้ เส้นเชื่อมจะเชื่อมต่อเป็นชั้นๆ ไม่ข้ามชั้นจากชั้นอินพุตไปชั้นฮิดเดน ถ้ามีชั้นฮิดเดนมากกว่าหนึ่งชั้นก็เชื่อมต่อกันไป และสุดท้ายจากชั้นฮิดเดนไปชั้นเอาต์พุต ข่ายงานป้อนไปหน้าแบบหลายชั้นนี้จะไม่มีเส้นเชื่อมย้อนกลับจะมีแต่เส้นเชื่อมไปข้างหน้าอย่างเดียวเช่นไม่มีเส้นเชื่อมจากบัพในชั้นเอาต์พุตส่งกลับมายังบัพในชั้นฮิดเดนหรือชั้นอินพุต เป็นต้น

ในการปรับค่าเวกเตอร์น้ำหนักโดยอัลกอริทึมการแพร่กระจายย้อนกลับนั้น เราต้องนิยามค่าผิดพลาดการสอนสำหรับข่ายงาน $E(\vec{w})$ จากนั้นจะหาค่าเวกเตอร์น้ำหนักที่ให้ค่าผิดพลาดต่ำสุด นิยามค่าผิดพลาดดังนี้

$$E(\vec{w}) = \frac{1}{2}\sum_{d\in D}\ \sum_{k\in outputs}(t_{kd} - o_{kd})^2 \qquad (6.14)$$

โดยที่ *outputs* คือเซตของบัพเอาต์พุตในข่ายงาน t_{kd} และ o_{kd} เป็นค่าเอาต์พุตเป้าหมายและเอาต์พุตที่ได้จากข่ายงานตามลำดับของบัพเอาต์พุตที่ k ของตัวอย่างตัวที่ d อัลกอริทึมการแพร่กระจายย้อนกลับจะค้นหาเวกเตอร์น้ำหนักที่ให้ค่าผิดพลาดต่ำสุด แต่ในกรณีของข่ายงานป้อนไปหน้าแบบหลายชั้นนี้ค่าต่ำสุดมักมีมากกว่าหนึ่งจุด ดังนั้นคำตอบของการแพร่กระจายย้อนกลับจึงเป็นค่าต่ำสุดเฉพาะที่ อัลกอริทึมแสดงในตารางที่ 6–23